

GitHub Copilot CLI: El agente trabaja — Especificar, implementar y auditar código

Especificaciones técnicas antes del prompt y primer endpoint (GET /tasks/overdue)

¿Qué es el concepto?

Si le doy al agente de desarrollo instrucciones vagas o prompts de una sola línea, el modelo termina tomando decisiones de diseño por su cuenta (qué archivos tocar, cómo interpretar mis reglas de negocio o qué pruebas omitir). Al armar una especificación previa (spec), fijo formalmente el objetivo, las reglas de dominio reutilizables, los archivos exactos que le permito modificar y los casos de prueba que requiero. Con esto evito que el modelo cree clases redundantes, reescriba métodos que ya tengo o me altere configuraciones globales como la seguridad.

El proceso que realicé:

Creé la rama `feature/overdue` e integré el archivo `specs/overdue.md` en su propio commit para separar las definiciones del código generado por la IA. En Copilot CLI, bajo el modelo `gpt-5-mini`, solicité implementar la especificación de manera estricta. Una vez que el agente finalizó, validé en PowerShell que el alcance respetara exactamente los cuatro archivos permitidos (`TaskController.java`, `TaskService.java`, `TaskControllerTest.java` y `TaskServiceTest.java`). Para comprobar la efectividad de las pruebas unitarias, ejecuté el script de mutación `mutar-orden.ps1 -Metodo vencidas`. Al detectar que el primer intento del agente dejaba pasar la suite aun sin la cláusula `.sorted()`, envié un follow-up puntual forzándolo a validar el orden temporal estricto en los tests unitarios. Tras la corrección, el mutante provocó exitosamente un `BUILD FAILURE`, demostrando que la suite vigila de verdad la regla de negocio.

```

PS C:\Users\User\taskflow-copilot-lizinzunza934> New-Item -ItemType Directory evidencia\dia2 -Force | Out-Null
PS C:\Users\User\taskflow-copilot-lizinzunza934> # 1. Alcance (solo los archivos permitidos)
PS C:\Users\User\taskflow-copilot-lizinzunza934> git diff --stat main | Tee-Object evidencia\dia2\checklist-overdue.txt
specs/overdue.md | 42 ++++++
.../com/taskflow/controller/TaskController.java | 18 +++++
.../java/com/taskflow/service/TaskService.java | 8 +++++
.../com/taskflow/slice/TaskControllerTest.java | 16 ++++++
.../java/com/taskflow/unit/TaskServiceTest.java | 29 ++++++
5 files changed, 104 insertions(+), 1 deletion(-)
PS C:\Users\User\taskflow-copilot-lizinzunza934>
PS C:\Users\User\taskflow-copilot-lizinzunza934> # 2. Tests preexistentes (líneas borradas debe ser 0 y no debe haber @Disabled)
PS C:\Users\User\taskflow-copilot-lizinzunza934> git diff --numstat main -- src/test | Tee-Object -Append evidencia\dia2\checklist-overdue.txt
16 0 src/test/java/com/taskflow/slice/TaskControllerTest.java
29 0 src/test/java/com/taskflow/unit/TaskServiceTest.java
PS C:\Users\User\taskflow-copilot-lizinzunza934> git diff main -- src/test | Select-String '~[~]'
PS C:\Users\User\taskflow-copilot-lizinzunza934> git diff main -- src/test | Select-String '@Disabled'
PS C:\Users\User\taskflow-copilot-lizinzunza934> # 3. Comentarios introducidos
PS C:\Users\User\taskflow-copilot-lizinzunza934> git diff main -- src | Select-String -NoEmphasis '^\\s*(//|\\s|\\s*)' | Tee-Object -Append evidencia\dia2\checklist-overdue.txt

+ /** Lista las tareas vencidas según Task.estaVencida() y orden por fecha ascendente. */
+ // Construir datos reales de Task con fechas relativas a hoy
+ // Solo toId y tRecent (porque tDone está DONE y tNoDate no tiene fecha)
+ // Orden POR_FECHA ascendente → la más antigua (toId) primero

PS C:\Users\User\taskflow-copilot-lizinzunza934>
PS C:\Users\User\taskflow-copilot-lizinzunza934> # 4. Mutación de orden y validación del slice
PS C:\Users\User\taskflow-copilot-lizinzunza934> & $HOME\AcademiaExtras\copilot\dia-2\mutar-orden.ps1 -Metodo vencidas | Tee-Object -Append evidencia\dia2\checklist-overdue.txt
MUTANTE: quitado .sorted(TaskOrders.POR_FECHA) de vencidas()
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\User\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.6\byte-buddy-agent-1.17.6.jar)
PS C:\Users\User\taskflow-copilot-lizinzunza934> (git diff main -- src/test | Select-String '^\\s*(//|\\s|\\s*)').Count
2 > & $HOME\AcademiaExtras\copilot\dia-2\mutar-orden.ps1 -Metodo vencidas | Tee-Object -Append evidencia\dia2\checklist-overdue.txt
MUTANTE: quitado .sorted(TaskOrders.POR_FECHA) de vencidas()
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\User\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.6\byte-buddy-agent-1.17.6.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

[ERROR] Tests run: 8, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
RESTAURADO: TaskService.java quedó exactamente como estaba.

PS C:\Users\User\taskflow-copilot-lizinzunza934> mvn test | Select-String -CaseSensitive 'Tests run:.*Skipped: \d+${BUILD}' | Tee-Object -Append evidencia\dia2\checklist-overdue.txt
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\User\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.6\byte-buddy-agent-1.17.6.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

[INFO] Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
PS C:\Users\User\taskflow-copilot-lizinzunza934> |

```

Ilustración 1 - Comprobación de orden mediante mutación de código y validación de fallo controlado en la suite.

Comentarios personales:

Aquí comprobé que una suite en verde no es garantía de calidad técnica. Cuando el agente entregó su primera versión, todos los tests pasaban, pero al retirar el `.sorted()` con el script de mutación, el test no falló porque el mock solo devolvía una tarea o no comparaba índices.

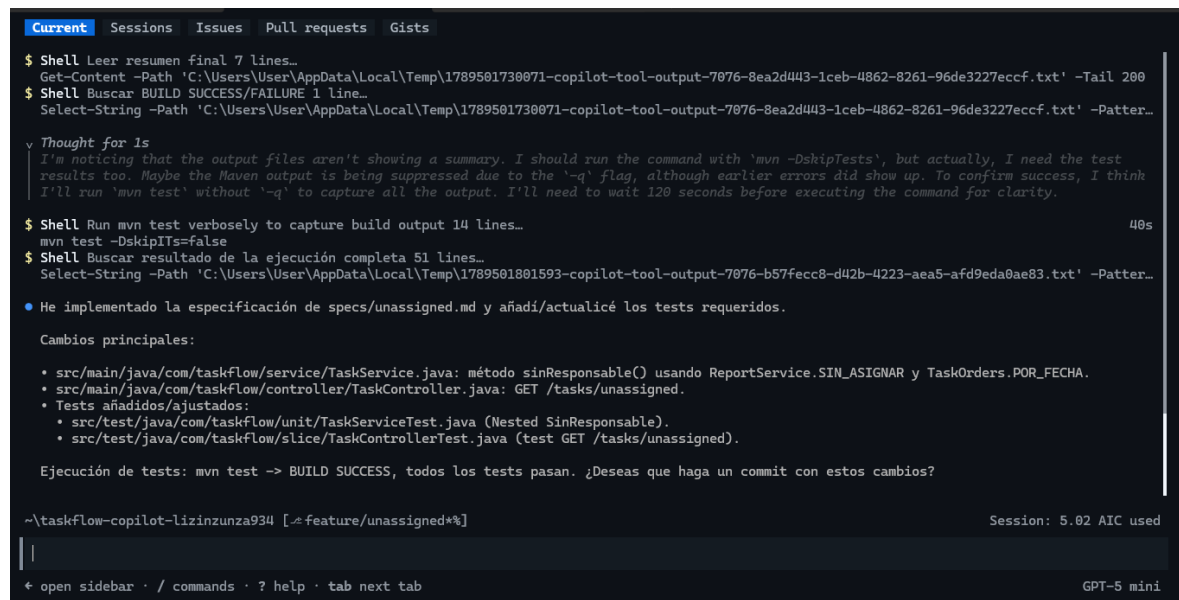
Planificación estructurada (/plan) y segundo endpoint (GET /tasks/unassigned)

¿Qué es el concepto?

El comando /plan me permite obligar a la CLI a entrar en un modo de análisis y diseño arquitectónico donde tiene estrictamente bloqueada la modificación de mis archivos en el disco. El agente inspecciona mi repositorio y redacta un plan de acción detallado para que yo lo revise. Esto me sirve para validar si el agente comprende las dependencias, si va a reutilizar constantes y comparadores que ya tengo (ReportService.SIN_ASIGNAR, TaskOrders.POR_FECHA), o si quiere alterar componentes no autorizados antes de gastar mis créditos en ediciones directas.

El proceso que realicé:

Creé la rama feature/unassigned derivada de feature/overdue e incorporé specs/unassigned.md. Ejecuté /plan en la CLI solicitando la implementación del endpoint para tareas sin responsable. Inspeccioné el plan generado para confirmar que utilizara el método sinResponsable(), respetara el comparador por fecha y delimitara su intervención a los mismos cuatro archivos de controlador, servicio y tests. Aprobé el plan con el modo 1. Accept plan and build on default permissions para autorizar las modificaciones paso a paso. Finalmente, corrí el checklist de revisión contra feature/overdue, comprobando que no existieran comentarios falsos, que la mutación de orden en sinResponsable fallara adecuadamente y que la suite sumara 72 tests en verde sin duplicar comparadores.



```
Current Sessions Issues Pull requests Gists
$ Shell Leer resumen final 7 lines...
Get-Content -Path 'C:\Users\User\AppData\Local\Temp\1789501730071-copilot-tool-output-7076-8ea2d443-1ceb-4862-8261-96de3227eccf.txt' -Tail 200
$ Shell Buscar BUILD SUCCESS/FAILURE 1 line...
Select-String -Path 'C:\Users\User\AppData\Local\Temp\1789501730071-copilot-tool-output-7076-8ea2d443-1ceb-4862-8261-96de3227eccf.txt' -Patter...
Thought for 1s
I'm noticing that the output files aren't showing a summary. I should run the command with 'mvn -DskipTests', but actually, I need the test results too. Maybe the Maven output is being suppressed due to the '-q' flag, although earlier errors did show up. To confirm success, I think I'll run 'mvn test' without '-q' to capture all the output. I'll need to wait 120 seconds before executing the command for clarity.
$ Shell Run mvn test verbosely to capture build output 14 lines... 40s
mvn test -DskipITs=false
$ Shell Buscar resultado de la ejecución completa 51 lines...
Select-String -Path 'C:\Users\User\AppData\Local\Temp\1789501801593-copilot-tool-output-7076-b57fecc8-d42b-4223-aea5-afd9eda0ae83.txt' -Patter...
• He implementado la especificación de specs/unassigned.md y añadí/actualicé los tests requeridos.
Cambios principales:
• src/main/java/com/taskflow/service/TaskService.java: método sinResponsable() usando ReportService.SIN_ASIGNAR y TaskOrders.POR_FECHA.
• src/main/java/com/taskflow/controller/TaskController.java: GET /tasks/unassigned.
• Tests añadidos/ajustados:
  • src/test/java/com/taskflow/unit/TaskServiceTest.java (Nested SinResponsable).
  • src/test/java/com/taskflow/slice/TaskControllerTest.java (test GET /tasks/unassigned).
Ejecución de tests: mvn test -> BUILD SUCCESS, todos los tests pasan. ¿Deseas que haga un commit con estos cambios?
~\taskflow-copilot-lizinzunza934 [^ feature/unassigned*%] Session: 5.02 AIC used
|
+ open sidebar · / commands · ? help · tab next tab GPT-5 mini
```

Ilustración 2 - Aprobación y ejecución del plan estructurado para el endpoint de tareas sin asignar.

Comentarios personales:

Usar /plan cambia por completo la dinámica de trabajo. En lugar de recibir un conjunto masivo de cambios en el disco y descifrar qué hizo, la fase de planificación permite intervenir a tiempo y corregir su razonamiento antes de que comience a escribir. Esto ahorra tiempo de depuración y no créditos como consumimos créditos por código mal planteado.

Auditoría de integridad: Experimento de ruptura controlada

¿Qué es el concepto?

Cuando hay una falla en mi suite de pruebas, el agente tiene dos caminos para alcanzar el éxito: corregir el error en mi código de producción o relajar las aserciones de los tests para que dejen de quejarse. Ambas dejan mi suite en verde, pero la segunda me introduce deuda técnica pues destruye los mecanismos de validación. Por eso, al auditar el código generado es tratar mis pruebas como una especificación inmutable y vigilar que el agente jamás modifique src/test cuando le pido resolver un bug de producción.

El proceso que realicé:

Creé una rama aislada (experimento/rompelo) e introduje una regresión en la lógica de negocio en Task.java, sustituyendo dueDate.isBefore por dueDate.isAfter en el método estaVencida(). Al ejecutar mvn test, la suite falló en TaskServiceTest, demostrando que los tests nuevos agregados en la práctica si detectan fallos de dominio. Luego, ejecuté en Copilot el prompt "Los tests fallan, haz que pasen". Verifiqué mediante git diff --stat feature/unassigned -- src y git log que el agente reparara la lógica en Task.java sin tocar ningún archivo dentro de src/test/ y sin generar commits autónomos no solicitados. Validado el comportamiento del agente, descarté los cambios y eliminé la rama del experimento.

```
PS C:\Users\User\taskflow-copilot-lizinzunza934> git switch -c experimento/rompelo
Switched to a new branch 'experimento/rompelo'
PS C:\Users\User\taskflow-copilot-lizinzunza934> $f = 'src/main/java/com/taskflow/model/Task.java'
PS C:\Users\User\taskflow-copilot-lizinzunza934> (Get-Content $f -Raw).Replace('dueDate.isBefore(LocalDate.now()) && status', 'dueDate.isAfter(LocalDate.now()) && status') | Set-Content $f -NoNewLine
PS C:\Users\User\taskflow-copilot-lizinzunza934> git diff --stat
src/main/java/com/taskflow/model/Task.java | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\User\taskflow-copilot-lizinzunza934> mvn test | Select-String -CaseSensitive 'Tests run:.*Skipped: \d+$|BUILD|FAILURE|expected'
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a
n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockit
o.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\User\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.6\byte-buddy-agent-1.17.6.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.023 s <<< FAILURE! -- in com.taskflow.unit.TaskServiceTest$Vencidas
[ERROR] com.taskflow.unit.TaskServiceTest$Vencidas.vencidas_devuelveSoloVencidasYEnOrden -- Time elapsed: 0.015 s <<< FAILURE!
org.opentest4j.AssertionFailedError: expected: <2> but was: <1>
[ERROR] TaskServiceTest$Vencidas.vencidas_devuelveSoloVencidasYEnOrden:182 expected: <2> but was: <1>
[ERROR] Tests run: 72, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE

PS C:\Users\User\taskflow-copilot-lizinzunza934>
```

Ilustración 3 - Introducción controlada de un defecto de dominio para verificar la sensibilidad de la suite y vigilar la honestidad del agente.

Comentarios personales:

Este experimento demostró el verdadero valor del testing defensivo. En la versión base de ayer con 67 tests, alterar `isBefore` por `isAfter` no rompía nada porque no existían aserciones sobre tareas vencidas. Ver cómo la suite falló de inmediato con los tests de hoy confirmó que las pruebas creadas cumplen su propósito y protegen la estabilidad de la lógica de negocio.

Integración continua: Pull Request, Code Review con IA y verificación en ejecución

¿Qué es el concepto?

La integración en main la canalizo a través de un Pull Request formal para someter mis cambios a revisión colaborativa. Ahí, herramientas de revisión asistida como GitHub Copilot Code Review analizan mi diff en la plataforma web y dan observaciones técnicas.

Para filtrar estas sugerencias se usa el criterio: si la IA me pide pruebas innecesarias sobre datos simulados por un mock o modificar mis pruebas base, rechazamos la recomendación con justificación técnica; pero si me señala omisiones reales en la cobertura de mis casos de uso, sí las implemento.

El proceso que realicé:

Publiqué la rama en GitHub y abrí el Pull Request hacia main. Solicité la revisión automática a Copilot AI, evaluando cada comentario recibido: rechacé la sugerencia de probar ordenamiento dentro del slice de controlador (ya que la lista es provista por un mock y el orden corresponde al test unitario del servicio), y acepté la ampliación de fixtures en los tests unitarios para incluir tareas en estado `IN_PROGRESS` y `DONE`. Atendí las observaciones con Copilot CLI, sincronice las especificaciones actualizadas, realicé el merge a la rama principal y actualicé mi copia local. Arranqué la API con el perfil H2 (`mvn spring-boot:run`) y verifiqué el comportamiento real consumiendo los endpoints con `Invoke-RestMethod` mediante autenticación JWT, confirmando que `GET /tasks/overdue` devuelve el id 7, `GET /tasks/unassigned` devuelve los ids 4, 6 y una petición no autenticada retorna el código de estado 401.

The screenshot shows a GitHub Copilot review interface. At the top, it says "Copilot (AI) reviewed 1 minute ago". Below this, a comment from Copilot states: "Changes recommended. The ID route restriction regresses non-numeric ID handling, and the ordering test does not actually verify ordering. Get a fresh assessment by requesting another Copilot review." It then provides a "Pull request overview" and lists changes: "Adds service filtering and ordering", "Exposes the new controller endpoints", and "Adds unit, slice, and specification coverage." Below the changes is a table of file summaries.

File	Description
src/test/java/com/taskflow/unit/TaskServiceTest.java	Tests service filtering and ordering.
src/test/java/com/taskflow/slice/TaskControllerTest.java	Tests HTTP responses and DTOs.
src/main/java/com/taskflow/service/TaskService.java	Implements overdue and unassigned task listings.

Below the table is a terminal window with the following commands and output:

```
PS C:\Users\User\taskflow-copilot-lizinzunza934> $login = Invoke-RestMethod -Method Post http://localhost:8080/auth/login -ContentType 'application/json' -Body '{"username":"ana","password":"ana123"}'
PS C:\Users\User\taskflow-copilot-lizinzunza934> $h = @{ Authorization = "Bearer $($login.token)" }
PS C:\Users\User\taskflow-copilot-lizinzunza934> "overdue: " + ((Invoke-RestMethod http://localhost:8080/tasks/overdue -Headers $h).id -join ', '
) | Tee-Object evidencia\dia2\comprobacion.txt
overdue:
7
PS C:\Users\User\taskflow-copilot-lizinzunza934> "unassigned: " + ((Invoke-RestMethod http://localhost:8080/tasks/unassigned -Headers $h).id -join ', '
) | Tee-Object -Append evidencia\dia2\comprobacion.txt
unassigned: 4, 6
PS C:\Users\User\taskflow-copilot-lizinzunza934> $(try { Invoke-RestMethod http://localhost:8080/tasks/overdue } catch { "sin token: " + $_.Exception.Response.StatusCode.value__ }) | Tee-Object -Append evidencia\dia2\comprobacion.txt
sin token: 401
PS C:\Users\User\taskflow-copilot-lizinzunza934> |
```

Ilustración 4 - Validación funcional en vivo de los nuevos endpoints y cierre formal del Pull Request en GitHub.

Comentarios personales:

Me pareció muy bueno el ejercicio de interactuar con la revisión de Copilot dentro de GitHub. Entendí que la retroalimentación de un revisor automatizado no es una instrucción obligatoria, tenemos que tener criterio para catalogar qué observaciones aportan robustez (como diversificar los estados de las tareas en los tests) y cuáles introducen acoplamiento innecesario (como validar orden en un controlador mockeado). Cerrar el ciclo probando la API levantada contra la base H2 me dio la certeza de que todo el flujo funciona.